

React Native Interview Questions & Answers: Topic 5 — Forms & Input

Section 1: Controlled & Uncontrolled Components

Q1: What is a controlled component in React Native?

A: A controlled component is an input component (like `TextInput`) whose value is bound to and driven by React state using the `value` and `onChangeText` props. State serves as the single source of truth.

Q2: What is an uncontrolled component in React Native?

A: An uncontrolled component manages its own internal value within the native layer. Instead of driving its value through React state, you access its current value when needed using a React ref.

Q3: Why are controlled components generally preferred in React Native form handling?

A: They enforce a single source of truth, making instant validation, dynamic UI updates, masked formatting, conditional submit buttons, and multi-step form orchestration straightforward.

Q4: What is the main drawback of controlled components in performance-critical scenarios?

A: Every keystroke triggers a state update, causing a React component re-render and cross-bridge/JSI communication between JavaScript and the native text view.

Q5: How do `onChangeText` and `onChange` differ in React Native's `TextInput`?

A: `onChangeText` passes the updated text string directly to the callback function. `onChange` passes a full synthetic event object containing `nativeEvent.text`.

Q6: How do you set an initial value for an uncontrolled `TextInput`?

A: Use the `defaultValue` prop instead of the `value` prop.

Q7: What happens if you pass a `value` prop to a `TextInput` without an `onChangeText` handler?

A: The input becomes read-only. The user can type, but the displayed text will instantly reset to whatever string is locked in the value state prop.

Q8: How do you read the current value of an uncontrolled TextInput?

A: Attach a ref via `useRef()` to the `TextInput`, then access `inputRef.current` or register native event listeners like `onEndEditing`.

Q9: When would you choose an uncontrolled component over a controlled component in React Native?

A: For ultra-large forms with dozens of fields on lower-end devices where re-rendering on every single keystroke causes input lag, or when bridging legacy native form controls.

Q10: How can you optimize a controlled TextInput to prevent lag on older devices?

A: Isolate the `TextInput` into its own sub-component state so keystrokes re-render only that specific input rather than the entire screen, or use debounced/throttled state updates for parent handlers.

Q11: How does React Hook Form leverage uncontrolled inputs for performance in React Native?

A: React Hook Form uses refs to isolate input values from the main component tree, updating the underlying native input directly without triggering full-screen re-renders on every keystroke.

Q12: How do you reset all fields in a controlled form?

A: Reset the state values tied to those inputs back to their initial state values (e.g., empty strings or null).

Q13: What is the risk of keeping `value={undefined}` in a controlled component?

A: Passing `undefined` or `null` switches the component from controlled to uncontrolled behavior unexpectedly, which can lead to state synchronization bugs.

Q14: How do you implement input masking (e.g., credit card formatting) with controlled components?

A: Intercept the raw string in the `onChangeText` handler, format it (adding dashes or spaces), and pass the formatted string to `setState`.

Q15: Can a component switch from uncontrolled to controlled during its lifecycle?

A: Yes, if value starts as undefined and later becomes a string. React will log a warning because this unpredictably changes how state synchronization works.

Q16: What is the role of `useRef` when focusing a `TextInput` programmatically?

A: `useRef` holds a direct reference to the native `TextInput` node, allowing you to execute native methods like `inputRef.current.focus()` or `inputRef.current.blur()`.

Q17: How do you move focus from the "First Name" input to "Last Name" when pressing "Next" on the keyboard?

A: Assign a ref to the Last Name input, set `returnKeyType="next"` on First Name, and invoke `lastNameRef.current.focus()` inside First Name's `onSubmitEditing` event.

Q18: What is `clear()` on a `TextInput` ref?

A: An imperative helper method on the native node that clears the text from the input immediately without manually setting state.

Q19: Why might `inputRef.current.clear()` break a controlled component's state?

A: Because it clears the native text view directly without notifying the React state, leading to a mismatch between state and what is displayed visually.

Q20: How do you handle numeric state in controlled inputs given `onChangeText` returns a string?

A: Convert the string to a number using `parseInt()` or `parseFloat()` when updating your numerical data model, or validate and sanitize non-numeric characters out of the string before updating state.

Q21: What is the single source of truth principle in form management?

A: The state holds the definitive data value at all times. The UI merely reflects that state and dispatches user actions to update it.

Q22: How do you programmatically blur a `TextInput`?

A: Obtain a reference using `useRef` and call `inputRef.current.blur()`.

Q23: Why does setting state on every keystroke sometimes drop characters on slow bridges?

A: Asynchronous bridge chatter between the JS thread sending state updates back to the UI thread can create a race condition where native keypresses arrive faster than the JS bridge responds.

Q24: How does the new React Native Architecture (Fabric) improve controlled component performance?

A: Fabric provides synchronous execution across JavaScript and C++ via the JSI, eliminating asynchronous bridge latency during high-frequency text inputs.

Q25: How do you manage multi-input form state efficiently with `useState`?

A: Use a single object state (e.g., `const [form, setForm] = useState({ email: "", password: "" })`) and update keys using computed property names: `setForm(prev => ({ ...prev, [key]: value }))`.

Q26: What is `onEndEditing` in `TextInput`?

A: A callback fired when the user finishes editing by submitting or moving focus away from the input.

Q27: How does `onEndEditing` differ from `onBlur`?

A: `onEndEditing` fires specifically when editing concludes (submit or blur), while `onBlur` fires whenever the input loses focus regardless of whether text was changed.

Q28: How do you capture both raw text and event metadata on input change?

A: Use the `onChange` prop instead of `onChangeText` to access `e.nativeEvent.text`, `e.nativeEvent.eventCount`, and target info.

Q29: What is the purpose of `eventCount` in `TextInput` native events?

A: It tracks the sequence number of events dispatched by the native input, enabling React to ignore out-of-order state synchronizations on the JS thread.

Q30: How can you perform real-time field validation without lag?

A: Validate the string inside the `onChangeText` callback before setting local state, or validate on `onBlur` to defer heavy calculations until input editing pauses.

Q31: How do you handle password visibility toggling in a controlled input?

A: Maintain a boolean state `isPasswordVisible` and pass `secureTextEntry={!isPasswordVisible}` to the `TextInput`.

Q32: Why should you avoid inline functions inside `onChangeText` for complex forms?

A: Inline functions create new function instances on every render. Extracted or memoized callbacks prevent unnecessary re-renders of parent or child components.

Q33: What is the difference between `defaultValue` and `value`?

A: `defaultValue` supplies an initial string for uncontrolled components. `value` forces the input string to stay strictly in sync with React state.

Q34: How do you force a controlled component to convert user input to uppercase in real time?

A: Inside `onChangeText`, transform the incoming text: `onChangeText={(text) => setText(text.toUpperCase())}`.

Q35: Can you turn an uncontrolled `TextInput` into a controlled one on demand?

A: While possible by passing a state variable to `value` dynamically, it is bad practice and causes unexpected component lifecycle warnings.

Section 2: TextInput Props & Validation Behaviors

Q36: What is the purpose of the keyboardType prop?

A: It determines which soft keyboard layout opens when the input gains focus (e.g., standard text, email address, phone keypad, decimal pad).

Q37: What are common values for keyboardType?

A: default, numeric, email-address, phone-pad, decimal-pad, number-pad, and url.

Q38: Does setting keyboardType="numeric" guarantee that users can only enter numbers?

A: No. Users can still paste non-numeric text from the clipboard, and some custom third-party keyboards bypass this prop. Sanitize the string in onChangeText.

Q39: What is the difference between keyboardType="number-pad" and keyboardType="numeric"?

A: number-pad shows digits 0–9 without punctuation. numeric may include decimal points, minus signs, or currency keys depending on OS and region.

Q40: What does the maxLength prop do?

A: Limits the maximum number of characters the user can enter directly into the text field.

Q41: Does maxLength restrict programmatic text assignment?

A: If you set a string longer than maxLength via state on a controlled input, the full string may render unless trimmed explicitly in JS.

Q42: What is the autoCorrect prop?

A: A boolean prop that turns auto-correction and spellchecking on (true) or off (false).

Q43: When should autoCorrect={false} be used?

A: On fields where non-standard dictionary terms are expected, such as usernames, passwords, promo codes, email addresses, and vehicle VIN numbers.

Q44: What are the possible options for autoCapitalize?

A: none, sentences (default), words, and characters.

Q45: How does autoCapitalize="words" behave?

A: It automatically capitalizes the first letter of each individual word (ideal for full names and address fields).

Q46: What is the returnKeyType prop?

A: Controls the visual label and icon displayed on the keyboard's "Return" or "Enter" key (e.g., done, go, next, search, send).

Q47: What platform constraints exist for returnKeyType values?

A: Some values are iOS-only (e.g., emergency-call, google, join, route) or Android-only (e.g., none). Universal cross-platform options include done, go, next, search, and send.

Q48: What does secureTextEntry={true} do?

A: Obscures typed characters (typically as dots or asterisks) for password entry.

Q49: Why does secureTextEntry sometimes cause font jumping on Android?

A: Android changes native font families dynamically when toggling text visibility. Fix this by explicitly defining a consistent fontFamily in the input's style.

Q50: How do you configure a TextInput for multi-line inputs?

A: Pass multiline={true}. You can also specify numberOfLines for Android or set custom height styling.

Q51: How does returnKeyType behave when multiline={true}?

A: On multiline inputs, pressing the Return key inserts a newline character (\n) by default rather than firing onSubmitEditing.

Q52: What is blurOnSubmit?

A: A boolean prop. If true, the text field loses focus and dismisses the keyboard when the user presses the keyboard's return button.

Q53: What is the default value of `blurOnSubmit`?

A: It defaults to true for single-line inputs and false for multiline inputs.

Q54: What does `placeholder` do?

A: Displays a light prompt string inside the input field when its value is empty.

Q55: How do you style placeholder text?

A: Use the `placeholderTextColor` prop to set color. Other text styles (font size, font family) inherit from the input's style object.

Q56: What does `selectTextOnFocus={true}` do?

A: Automatically highlights all existing text inside the field when it gains focus, allowing easy replacement.

Q57: What does `clearTextOnFocus={true}` do?

A: Automatically clears all text from the field upon focus (iOS only).

Q58: What is `onSubmitEditing`?

A: A callback invoked when the user hits the keyboard's submit/return button.

Q59: How do you auto-focus a `TextInput` as soon as a screen renders?

A: Set `autoFocus={true}` on the `TextInput` component.

Q60: Why should you avoid overusing `autoFocus` in complex screens?

A: Triggering multiple auto-focus operations or auto-focusing during screen transition animations can cause frame drops and layout jitter.

Q61: What prop prevents users from copying or pasting text inside a `TextInput`?

A: Set `contextMenuHidden={true}` to hide the native selection menu (Cut, Copy, Paste).

Q62: What is `editable` prop?

A: A boolean prop. When false, the input is disabled—users cannot tap, edit, or focus it.

Q63: How does `editable={false}` differ from `pointerEvents="none"`?

A: `editable={false}` native-disabled behavior allows touch interactions to pass through or trigger outer pressable containers, whereas `pointerEvents="none"` blocks touch events on the component wrapper level completely.

Q64: What is `caretHidden`?

A: A boolean prop that hides the flashing cursor when set to true.

Q65: How do you set custom cursor colors on iOS and Android?

A: Use the `selectionColor` prop.

Q66: What prop tracks selection range changes inside an input?

A: `onSelectionChange`, which returns `nativeEvent.selection.start` and `nativeEvent.selection.end`.

Q67: How do you prevent auto-filling sensitive fields like PIN codes?

A: Set `textContentType="none"` (iOS) or `autoComplete="off"` (Android).

Q68: What is `textContentType` on iOS?

A: A prop that hints to iOS Autofill and Password Manager what content type is expected (e.g., `oneTimeCode`, `username`, `password`, `emailAddress`).

Q69: What is the Android equivalent of `textContentType`?

A: `autoComplete` (e.g., `autoComplete="sms-otp"`, `autoComplete="email"`).

Q70: How do you handle dynamic font scaling issues inside fixed-height `TextInput` fields?

A: Either set `allowFontScaling={false}` or design inputs using `padding/minHeight` instead of hardcoded height.

Section 3: KeyboardAvoidingView & Keyboard Management

Q71: What is the purpose of KeyboardAvoidingView?

A: It automatically shifts, resizes, or offsets layout views when the virtual keyboard appears so that inputs aren't obscured.

Q72: What are the three primary values for the behavior prop in KeyboardAvoidingView?

A: padding, height, and position.

Q73: Why is platform branching typically necessary when configuring KeyboardAvoidingView?

A: iOS handles full-window frame resizing differently than Android. Typically, `behavior="padding"` works best on iOS, while Android often requires `behavior="height"` or no wrapper at all if handled natively in `AndroidManifest.xml`.

Q74: What is keyboardVerticalOffset?

A: A numerical prop specifying the distance between the top of the user screen and the React Native header/view, preventing inaccurate height recalculations.

Q75: How do you dismiss the keyboard when tapping outside an input field?

A: Wrap the screen container with `TouchableWithoutFeedback` and call `Keyboard.dismiss()` inside its `onPress` handler.

Q76: What is the Keyboard module in React Native?

A: A core API used to emit and listen to native keyboard state changes and dismiss the active keyboard programmatically.

Q77: How do you listen for keyboard display events in React Native?

A: Register listeners using `Keyboard.addListener('keyboardDidShow', callback)` and `Keyboard.addListener('keyboardDidHide', callback)`.

Q78: What is the difference between `keyboardWillShow` and `keyboardDidShow`?

A: `keyboardWillShow` fires at the start of the keyboard animation (iOS only). `keyboardDidShow` fires after the keyboard animation completes.

Q79: Why doesn't `keyboardWillShow` work on Android?

A: Android native APIs historically did not broadcast keyboard animation start frames to JavaScript; they only emit state once visible (`keyboardDidShow`).

Q80: How do you clean up Keyboard listeners in React functional components?

A: Store the subscription object and call `.remove()` inside the cleanup function of a `useEffect` hook.

```
JavaScript
useEffect(() => {
  const sub = Keyboard.addListener('keyboardDidShow', () => {});
  return () => sub.remove();
}, []);
```

Q81: What Android setting in `AndroidManifest.xml` dictates keyboard resizing behavior?

A: `android:windowSoftInputMode`. Setting it to `adjustResize` resizes the screen layout automatically, while `adjustPan` shifts the screen focus up.

Q82: What common bug occurs when pairing `KeyboardAvoidingView` with `adjustResize` on Android?

A: Double padding/shifting: both Android OS and `KeyboardAvoidingView` resize the viewport simultaneously, creating excess white space.

Q83: What prop on `ScrollView` dictates how taps are handled when the keyboard is active?

A: `keyboardShouldPersistTaps`.

Q84: What are the valid values for `keyboardShouldPersistTaps`?

A: 'never' (default), 'always', and 'handled'.

Q85: What happens when `keyboardShouldPersistTaps="never"`?

A: Tapping outside the input while the keyboard is up dismisses the keyboard; child touch events (e.g., tapping a submit button) are ignored on that first tap.

Q86: Why is `keyboardShouldPersistTaps="handled"` recommended for forms inside `ScrollView`?

A: It allows buttons to fire their `onPress` event immediately on the first tap while automatically dismissing the keyboard if the tap misses interactable targets.

Q87: What is `keyboardDismissMode` on `ScrollView`?

A: Determines whether scrolling dismisses the keyboard automatically. Options include 'none', 'on-drag', and 'interactive' (iOS).

Q88: Why does `KeyboardAvoidingView` sometimes fail inside flexible layouts (`flex: 1`)?

A: If parent containers lack explicit `flex: 1` sizing, `KeyboardAvoidingView` cannot compute correct dynamic height deltas during resize events.

Q89: How do third-party packages like `react-native-keyboard-aware-scroll-view` work?

A: They dynamically calculate the absolute layout coordinates of focused inputs and scroll the container smoothly to ensure the active field stays centered above the keyboard.

Q90: How do you determine the height of the virtual keyboard dynamically?

A: Listen to `keyboardDidShow` events; the payload contains `e.endCoordinates.height`.

Q91: Why is `TouchableWithoutFeedback` preferred over `TouchableOpacity` for dismissing keyboards?

A: `TouchableWithoutFeedback` does not alter opacity or provide visual feedback, making it imperceptible as a background tap handler.

Q92: What issue occurs when placing `KeyboardAvoidingView` inside a `ScrollView` instead of wrapping it?

A: Layout calculations conflict: the `ScrollView` attempts to measure content height while `KeyboardAvoidingView` compresses the outer viewport, causing flickering.

Q93: How do bottom sheet modal inputs handle keyboard avoidance?

A: Bottom sheets typically require custom vertical offsets, platform-specific listener height adjustments, or native modal flags (`statusBarTranslucent`).

Q94: How do you keep a fixed sticky submit button floating right above the open keyboard?

A: Place the button container outside the `ScrollView` but inside a `KeyboardAvoidingView` with `behavior="padding"`.

Q95: What is `Keyboard.dismiss()`?

A: A helper method that explicitly hides the active virtual keyboard and clears input focus across the screen.

Q96: What is the benefit of `behavior="position"` in `KeyboardAvoidingView`?

A: Instead of resizing views or squeezing layouts, it translates the position of child views upward, keeping view dimensions intact.

Q97: What is a drawback of `behavior="position"`?

A: It can push header bars, navigation components, or status bar elements off the top edge of the visible screen.

Q98: How do custom input accessories (e.g., a "Done" button toolbar above the numeric keypad) work on iOS?

A: You can render an `InputAccessoryView` linked to the input via an `inputAccessoryViewID` prop.

Q99: Why doesn't `InputAccessoryView` work on Android natively?

A: Android does not support native keyboard accessory bars attached above soft keypads; custom UI bars must be positioned manually in JS using keyboard listeners.

Q100: What is the golden rule for troubleshooting KeyboardAvoidingView layout breakages?

A: Ensure all top-level parent view wrappers have flex: 1, verify keyboardVerticalOffset accounts for navigation bars, and apply platform-specific behavior settings (padding for iOS, no behavior or height for Android).